

CO4 - AT1 comparative Analysis

1) Hashing vs sorting - Based search (Binary search)

Hashing

1) Avg search $O(1)$ worst case: $O(n)$ due to collisions

2) Requires extra memory for the hash table $O(n)$

3) Fast insertion & deletion avg $O(1)$. Suitable for frequent changing data

4) Used in databases, caches, dictionaries, symbol tables, hash maps

Binary search

1) $O(\log n)$ Data must be sorted first

2) Binary search itself needs $O(1)$ extra space for array

3) Insertion can be $O(n)$ because elements may need to be shifted to maintain sorted order

4) Used when data is static, such as searching sorted arrays.

• For a dynamic system with frequent insertion, deletions & searches, hashing is generally better, because it provides average $O(1)$ search, insertion, deletion. It provides $O(\log n)$ maintaining sorted data.

2) Quick sort vs Merge sort

Quick sort

1) Best case $O(n \log n)$

2) Avg case $O(n \log n)$

3) Worst case $O(n^2)$

4) Generally not stable

5) usually $O(\log n)$ auxiliary space due to recursion

6) usually very fast

Merge sort

1) $O(n \log n)$

2) $O(n \log n)$

3) $O(n \log n)$

4) stable

5) Requires $O(n)$ extra space for merging

6) Predictable performance

Quick sort is often preferred

Although Quick sort has a worst-case complexity of $O(n^2)$ it is often preferred.

1) It is avg-case complexity is $O(n \log n)$

2) It is generally required less additional memory than merge sort

3) It has good cache locality because it mainly operation within same array